

code/ecf/shellex.c

```

1  /* parseline - Parse the command line and build the argv array */
2  int parseline(char *buf, char **argv)
3  {
4      char *delim;          /* Points to first space delimiter */
5      int argc;             /* Number of args */
6      int bg;               /* Background job? */
7
8      buf[strlen(buf)-1] = ' '; /* Replace trailing '\n' with space */
9      while (*buf && (*buf == ' ')) /* Ignore leading spaces */
10         buf++;
11
12     /* Build the argv list */
13     argc = 0;
14     while ((delim = strchr(buf, ' '))) {
15         argv[argc++] = buf;
16         *delim = '\0';
17         buf = delim + 1;
18         while (*buf && (*buf == ' ')) /* Ignore spaces */
19             buf++;
20     }
21     argv[argc] = NULL;
22
23     if (argc == 0) /* Ignore blank line */
24         return 1;
25
26     /* Should the job run in the background? */
27     if ((bg = (*argv[argc-1] == '&')) != 0)
28         argv[--argc] = NULL;
29
30     return bg;
31 }

```

code/ecf/shellex.c

图 8-25 parseline 解析 shell 的一个输入行

注意，这个简单的 shell 是有缺陷的，因为它并不回收它的后台子进程。修改这个缺陷就要求使用信号，我们将在下一节中讲述信号。

8.5 信号

到目前为止对异常控制流的学习中，我们已经看到了硬件和软件是如何合作以提供基本的低层异常机制的。我们也看到了操作系统如何利用异常来支持进程上下文切换的异常控制流形式。在本节中，我们将研究一种更高层的软件形式的异常，称为 Linux 信号，它允许进程和内核中断其他进程。

一个信号就是一条小消息，它通知进程系统中发生了一个某种类型的事件。比如，图 8-26 展示了 Linux 系统上支持的 30 种不同类型的信号。

每种信号类型都对应于某种系统事件。低层的硬件异常是由内核异常处理程序处理的，正常情况下，对用户进程而言是不可见的。信号提供了一种机制，通知用户进程发生了这些异常。比如，如果一个进程试图除以 0，那么内核就发送给它一个 SIGFPE 信号(号码 8)。如果一个进

程执行一条非法指令,那么内核就发送给它一个 SIGILL 信号(号码 4)。如果进程进行非法内存引用,内核就发送给它一个 SIGSEGV 信号(号码 11)。其他信号对应于内核或者其他用户进程中较高层的软件事件。比如,如果当进程在前台运行时,你键入 Ctrl+C(也就是同时按下 Ctrl 键和 C 键),那么内核就会发送一个 SIGINT 信号(号码 2)给这个前台进程组中的每个进程。一个进程可以通过向另一个进程发送一个 SIGKILL 信号(号码 9)强制终止它。当一个子进程终止或者停止时,内核会发送一个 SIGCHLD 信号(号码 17)给父进程。

序号	名称	默认行为	相应事件
1	SIGHUP	终止	终端线挂断
2	SIGINT	终止	来自键盘的中断
3	SIGQUIT	终止	来自键盘的退出
4	SIGILL	终止	非法指令
5	SIGTRAP	终止并转储内存 ^①	跟踪陷阱
6	SIGABRT	终止并转储内存 ^①	来自 abort 函数的终止信号
7	SIGBUS	终止	总线错误
8	SIGFPE	终止并转储内存 ^①	浮点异常
9	SIGKILL	终止 ^②	杀死程序
10	SIGUSR1	终止	用户定义的信号 1
11	SIGSEGV	终止并转储内存 ^①	无效的内存引用(段故障)
12	SIGUSR2	终止	用户定义的信号 2
13	SIGPIPE	终止	向一个没有读用户的管道做写操作
14	SIGALRM	终止	来自 alarm 函数的定时器信号
15	SIGTERM	终止	软件终止信号
16	SIGSTKFLT	终止	协处理器上的栈故障
17	SIGCHLD	忽略	一个子进程停止或者终止
18	SIGCONT	忽略	继续进程如果该进程停止
19	SIGSTOP	停止直到下一个 SIGCONT ^②	不是来自终端的停止信号
20	SIGTSTP	停止直到下一个 SIGCONT	来自终端的停止信号
21	SIGTTIN	停止直到下一个 SIGCONT	后台进程从终端读
22	SIGTTOU	停止直到下一个 SIGCONT	后台进程向终端写
23	SIGURG	忽略	套接字上的紧急情况
24	SIGXCPU	终止	CPU 时间限制超出
25	SIGXFSZ	终止	文件大小限制超出
26	SIGVTALRM	终止	虚拟定时器期满
27	SIGPROF	终止	剖析定时器期满
28	SIGWINCH	忽略	窗口大小变化
29	SIGIO	终止	在某个描述符上可执行 I/O 操作
30	SIGPWR	终止	电源故障

图 8-26 Linux 信号

注:①多年前,主存是用一种称为磁芯存储器(core memory)的技术来实现的。“转储内存”(dumping core)是一个历史术语,意思是把代码和数据内存段的映像写到磁盘上。

②这个信号既不能被捕获,也不能被忽略。

(来源:man 7 signal。数据来自 Linux Foundation。)

8.5.1 信号术语

传送一个信号到目的进程是由两个不同步骤组成的:

- 发送信号。内核通过更新目的进程上下文中的某个状态,发送(递送)一个信号给目的进程。发送信号可以有如下两种原因:1)内核检测到一个系统事件,比如除零错误或者子进程终止。2)一个进程调用了 kill 函数(在下一节中讨论),显式地要求内核发送一个信号给目的进程。一个进程可以发送信号给自己。

- 接收信号。当目的进程被内核强迫以某种方式对信号的发送做出反应时，它就接收了信号。进程可以忽略这个信号，终止或者通过执行一个称为信号处理程序(signal handler)的用户层函数捕获这个信号。图 8-27 给出了信号处理程序捕获信号的基本思想。

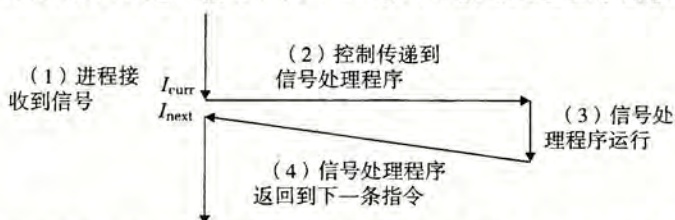


图 8-27 信号处理。接收到信号会触发控制转移到信号处理程序。在信号处理程序完成处理之后，它将控制返回给被中断的程序

一个发出而没有被接收的信号叫做待处理信号(pending signal)。在任何时刻，一种类型至多只会会有一个待处理信号。如果一个进程有一个类型为 k 的待处理信号，那么任何接下来发送到这个进程的类型为 k 的信号都不会排队等待；它们只是被简单地丢弃。一个进程可以有选择性地阻塞接收某种信号。当一种信号被阻塞时，它仍可以被发送，但是产生的待处理信号不会被接收，直到进程取消对这种信号的阻塞。

一个待处理信号最多只能被接收一次。内核为每个进程在 pending 位向量中维护着待处理信号的集合，而在 blocked 位向量[⊖]中维护着被阻塞的信号集合。只要传送了一个类型为 k 的信号，内核就会设置 pending 中的第 k 位，而只要接收了一个类型为 k 的信号，内核就会清除 pending 中的第 k 位。

8.5.2 发送信号

Unix 系统提供了大量向进程发送信号的机制。所有这些机制都是基于进程组(process group)这个概念的。

1. 进程组

每个进程都只属于一个进程组，进程组是由一个正整数进程组 ID 来标识的。getpgrp 函数返回当前进程的进程组 ID：

```
#include <unistd.h>

pid_t getpgrp(void);
```

返回：调用进程的进程组 ID。

默认地，一个子进程和它的父进程同属于一个进程组。一个进程可以通过使用 setpgid 函数来改变自己或者其他进程的进程组：

```
#include <unistd.h>

int setpgid(pid_t pid, pid_t pgid);
```

返回：若成功则为 0，若错误则为 -1。

setpgid 函数将进程 pid 的进程组改为 pgid。如果 pid 是 0，那么就使用当前进程

[⊖] 也称为信号掩码(signal mask)。

的 PID。如果 pgid 是 0，那么就用 pid 指定的进程的 PID 作为进程组 ID。例如，如果进程 15213 是调用进程，那么

```
setpgid(0, 0);
```

会创建一个新的进程组，其进程组 ID 是 15213，并且把进程 15213 加入到这个新的进程组中。

2. 用/bin/kill 程序发送信号

/bin/kill 程序可以向另外的进程发送任意的信号。比如，命令

```
linux> /bin/kill -9 15213
```

发送信号 9(SIGKILL)给进程 15213。一个为负的 PID 会导致信号被发送到进程组 PID 中的每个进程。比如，命令

```
linux> /bin/kill -9 -15213
```

发送一个 SIGKILL 信号给进程组 15213 中的每个进程。注意，在此我们使用完整路径/bin/kill，因为有些 Unix shell 有自己内置的 kill 命令。

3. 从键盘发送信号

Unix shell 使用作业(job)这个抽象概念来表示为对一条命令行求值而创建的进程。在任何时刻，至多只有一个前台作业和 0 个或多个后台作业。比如，键入

```
linux> ls | sort
```

会创建一个由两个进程组成的前台作业，这两个进程是通过 Unix 管道连接起来的：一个进程运行 ls 程序，另一个运行 sort 程序。shell 为每个作业创建一个独立的进程组。进程组 ID 通常取自作业中父进程中的一个。比如，图 8-28 展示了有一个前台作业和两个后台作业的 shell。前台作业中的父进程 PID 为 20，进程组 ID 也为 20。父进程创建两个子进程，每个也都是进程组 20 的成员。

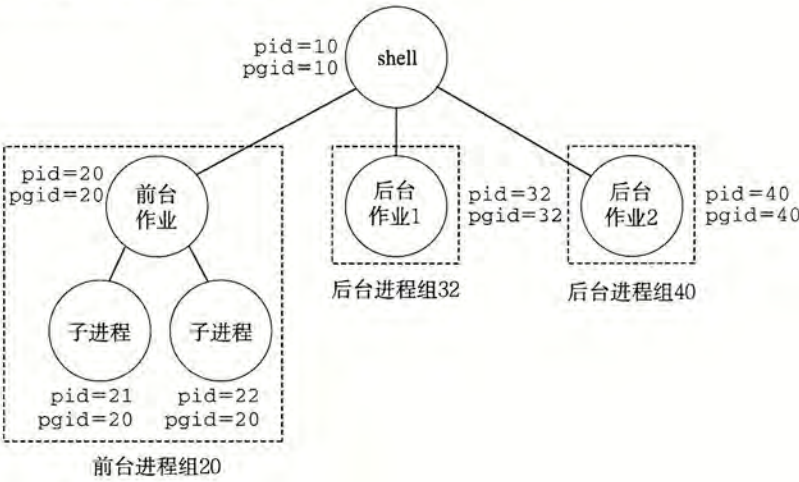


图 8-28 前台和后台进程组

在键盘上输入 Ctrl+C 会导致内核发送一个 SIGINT 信号到前台进程组中的每个进程。默认情况下，结果是终止前台作业。类似地，输入 Ctrl+Z 会发送一个 SIGTSTP 信号到前台进程组中的每个进程。默认情况下，结果是停止(挂起)前台作业。

4. 用 kill 函数发送信号

进程通过调用 kill 函数发送信号给其他进程(包括它们自己)。

```
#include <sys/types.h>
#include <signal.h>

int kill(pid_t pid, int sig);
```

返回：若成功则为 0，若错误则为 -1。

如果 pid 大于零，那么 kill 函数发送信号号码 sig 给进程 pid。如果 pid 等于零，那么 kill 发送信号 sig 给调用进程所在进程组中的每个进程，包括调用进程自己。如果 pid 小于零，kill 发送信号 sig 给进程组 |pid|(pid 的绝对值)中的每个进程。图 8-29 展示了一个示例，父进程用 kill 函数发送 SIGKILL 信号给它的子进程。

```

1  #include "csapp.h"
2
3  int main()
4  {
5      pid_t pid;
6
7      /* Child sleeps until SIGKILL signal received, then dies */
8      if ((pid = Fork()) == 0) {
9          Pause(); /* Wait for a signal to arrive */
10         printf("control should never reach here!\n");
11         exit(0);
12     }
13
14     /* Parent sends a SIGKILL signal to a child */
15     Kill(pid, SIGKILL);
16     exit(0);
17 }
```

code/ecf/kill.c

code/ecf/kill.c

图 8-29 使用 kill 函数发送信号给子进程

5. 用 alarm 函数发送信号

进程可以通过调用 alarm 函数向它自己发送 SIGALRM 信号。

```
#include <unistd.h>

unsigned int alarm(unsigned int secs);
```

返回：前一次闹钟剩余的秒数，若以前没有设定闹钟，则为 0。

alarm 函数安排内核在 secs 秒后发送一个 SIGALRM 信号给调用进程。如果 secs 是零，那么不会调度安排新的闹钟(alarm)。在任何情况下，对 alarm 的调用都将取消任何待处理的(pending)闹钟，并且返回任何待处理的闹钟在被发送前还剩下的秒数(如果这次对 alarm 的调用没有取消它的话)；如果没有任何待处理的闹钟，就返回零。

8.5.3 接收信号

当内核把进程 p 从内核模式切换到用户模式时(例如, 从系统调用返回或是完成了一次上下文切换), 它会检查进程 p 的未被阻塞的待处理信号的集合(pending & ~blocked)。如果这个集合为空(通常情况下), 那么内核将控制传递到 p 的逻辑控制流中的下一条指令(I_{next})。然而, 如果集合是非空的, 那么内核选择集合中的某个信号 k (通常是最小的 k), 并且强制 p 接收信号 k 。收到这个信号会触发进程采取某种行为。一旦进程完成了这个行为, 那么控制就传递回 p 的逻辑控制流中的下一条指令(I_{next})。每个信号类型都有一个预定义的默认行为, 是下面中的一种:

- 进程终止。
- 进程终止并转储内存。
- 进程停止(挂起)直到被 SIGCONT 信号重启。
- 进程忽略该信号。

图 8-26 展示了与每个信号类型相关联的默认行为。比如, 收到 SIGKILL 的默认行为就是终止接收进程。另外, 接收到 SIGCHLD 的默认行为就是忽略这个信号。进程可以通过使用 signal 函数修改和信号相关联的默认行为。唯一的例外是 SIGSTOP 和 SIGKILL, 它们的默认行为是不能修改的。

```
#include <signal.h>
typedef void (*sighandler_t)(int);

sighandler_t signal(int signum, sighandler_t handler);
```

返回: 若成功则为指向前次处理程序的指针, 若出错则为 SIG_ERR(不设置 errno)。

signal 函数可以通过下列三种方法之一来改变和信号 signum 相关联的行为:

- 如果 handler 是 SIG_IGN, 那么忽略类型为 signum 的信号。
- 如果 handler 是 SIG_DFL, 那么类型为 signum 的信号行为恢复为默认行为。
- 否则, handler 就是用户定义的函数的地址, 这个函数被称为信号处理程序, 只要进程接收到一个类型为 signum 的信号, 就会调用这个程序。通过把处理程序的地址传递到 signal 函数从而改变默认行为, 这叫做设置信号处理程序(installing the handler)。调用信号处理程序被称为捕获信号。执行信号处理程序被称为处理信号。

当一个进程捕获了一个类型为 k 的信号时, 会调用为信号 k 设置的处理程序, 一个整数参数被设置为 k 。这个参数允许同一个处理函数捕获不同类型的信号。

当处理程序执行它的 return 语句时, 控制(通常)传递回控制流中进程被信号接收中断位置处的指令。我们说“通常”是因为在某些系统中, 被中断的系统调用会立即返回一个错误。

图 8-30 展示了一个程序, 它捕获用户在键盘上输入 Ctrl+C 时发送的 SIGINT 信号。SIGINT 的默认行为是立即终止该进程。在这个示例中, 我们将默认行为修改为捕获信号, 输出一条消息, 然后终止该进程。

信号处理程序可以被其他信号处理程序中断, 如图 8-31 所示。在这个例子中, 主程序捕获到信号 s , 该信号会中断主程序, 将控制转移到处理程序 S 。 S 在运行时, 程序捕获信号 $t \neq s$, 该信号会中断 S , 控制转移到处理程序 T 。当 T 返回时, S 从它被中断的地方继续执行。最后, S 返回, 控制传送回主程序, 主程序从它被中断的地方继续执行。


```

code/ecf/sigint.c
1  #include "csapp.h"
2
3  void sigint_handler(int sig) /* SIGINT handler */
4  {
5      printf("Caught SIGINT!\n");
6      exit(0);
7  }
8
9  int main()
10 {
11     /* Install the SIGINT handler */
12     if (signal(SIGINT, sigint_handler) == SIG_ERR)
13         unix_error("signal error");
14
15     pause(); /* Wait for the receipt of a signal */
16
17     return 0;
18 }
code/ecf/sigint.c

```

图 8-30 一个用信号处理程序捕获 SIGINT 信号的程序

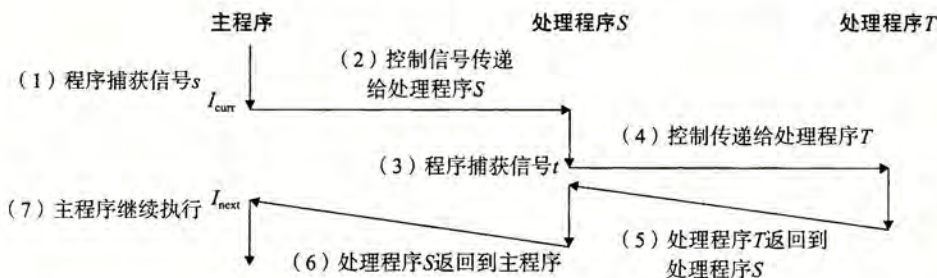


图 8-31 信号处理程序可以被其他信号处理程序中断

 **练习题 8.7** 编写一个叫做 `snooze` 的程序，它有一个命令行参数，用这个参数调用练习题 8.5 中的 `snooze` 函数，然后终止。编写程序，使得用户可以通过在键盘上输入 `Ctrl+C` 中断 `snooze` 函数。比如：

```

linux> ./snooze 5
CTRL+C                               User hits Ctrl+C after 3 seconds
Slept for 3 of 5 secs.
linux>

```

8.5.4 阻塞和解除阻塞信号

Linux 提供阻塞信号的隐式和显式的机制：

隐式阻塞机制。内核默认阻塞任何当前处理程序正在处理信号类型的待处理的信号。例如，图 8-31 中，假设程序捕获了信号 s ，当前正在运行处理程序 S 。如果发送给该进程另一个信号 s ，那么直到处理程序 S 返回， s 会变成待处理而没有被接收。

显式阻塞机制。应用程序可以使用 `sigprocmask` 函数和它的辅助函数，明确地阻塞和解除阻塞选定的信号。

```
#include <signal.h>
```

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
```

返回：如果成功则为 0，若出错则为 -1。

```
int sigismember(const sigset_t *set, int signum);
```

返回：若 signum 是 set 的成员则为 1，如果不是则为 0，若出错则为 -1。

sigprocmask 函数改变当前阻塞的信号集合(8.5.1 节中描述的 blocked 位向量)。具体的行为依赖于 how 的值：

SIG_BLOCK：把 set 中的信号添加到 blocked 中(blocked=blocked | set)。

SIG_UNBLOCK：从 blocked 中删除 set 中的信号(blocked=blocked &~set)。

SIG_SETMASK：block=set。

如果 oldset 非空，那么 blocked 位向量之前的值保存在 oldset 中。

使用下述函数对 set 信号集合进行操作：sigemptyset 初始化 set 为空集合。sigfillset 函数把每个信号都添加到 set 中。sigaddset 函数把 signum 添加到 set，sigdelset 从 set 中删除 signum，如果 signum 是 set 的成员，那么 sigismember 返回 1，否则返回 0。

例如，图 8-32 展示了如何用 sigprocmask 来临时阻塞接收 SIGINT 信号。

```
1      sigset_t mask, prev_mask;
2
3      Sigemptyset(&mask);
4      Sigaddset(&mask, SIGINT);
5
6      /* Block SIGINT and save previous blocked set */
7      Sigprocmask(SIG_BLOCK, &mask, &prev_mask);
8      : // Code region that will not be interrupted by SIGINT
9
10     /* Restore previous blocked set, unblocking SIGINT */
11     Sigprocmask(SIG_SETMASK, &prev_mask, NULL);
```

图 8-32 临时阻塞接收一个信号

8.5.5 编写信号处理程序

信号处理是 Linux 系统编程最棘手的一个问题。处理程序有几个属性使得它们很难推理分析：1) 处理程序与主程序并发运行，共享同样的全局变量，因此可能与主程序和其他处理程序互相干扰；2) 如何以及何时接收信号的规则常常有违人的直觉；3) 不同的系统有不同的信号处理语义。

在本节中，我们将讲述这些问题，介绍编写安全、正确和可移植的信号处理程序的一些基本规则。

1. 安全的信号处理

信号处理程序很麻烦是因为它们和主程序以及其他信号处理程序并发地运行，正如我们在图 8-31 中看到的那样。如果处理程序和主程序并发地访问同样的全局数据结构，那

么结果可能就不可预知，而且经常是致命的。

我们会在第 12 章详细讲述并发编程。这里我们的目标是给你一些保守的编写处理程序的原则，使得这些处理程序能安全地并发运行。如果你忽视这些原则，就可能有引入细微的并发错误的风险。如果有这些错误，程序可能在绝大部分时候都能正确工作。然而当它出错的时候，就会错得不可预测和不可重复，这样是很难调试的。一定要防患于未然！

- G0. 处理程序要尽可能简单。避免麻烦的最好方法是保持处理程序尽可能小和简单。例如，处理程序可能只是简单地设置全局标志并立即返回；所有与接收信号相关的处理都由主程序执行，它周期性地检查(并重置)这个标志。
- G1. 在处理程序中只调用异步信号安全的函数。所谓异步信号安全的函数(或简称安全的函数)能够被信号处理程序安全地调用，原因有二：要么它是可重入的(例如只访问局部变量，见 12.7.2 节)，要么它不能被信号处理程序中断。图 8-33 列出了 Linux 保证安全的系统级函数。注意，许多常见的函数(例如 printf、sprintf、malloc 和 exit)都不在此列。

_Exit	fexecve	poll	sigqueue
_exit	fork	posix_trace_event	sigset
abort	fstat	pselect	sigsuspend
accept	fstatat	raise	sleep
access	fsync	read	socketmark
aio_error	ftruncate	readlink	socket
aio_return	futimens	readlinkat	socketpair
aio_suspend	getegid	recv	stat
alarm	geteuid	recvfrom	symlink
bind	getgid	recvmsg	symlinkat
cfgetispeed	getgroups	rename	tcdrain
cfgetospeed	getpeername	renameat	tcflow
cfsetispeed	getpgrp	rmdir	tcflush
cfsetospeed	getpid	select	tcgetattr
chdir	getppid	sem_post	tcgetpgrp
chmod	getsockname	send	tcsendbreak
chown	getsockopt	sendmsg	tcsetattr
clock_gettime	getuid	sendto	tcsetpgrp
close	kill	setgid	time
connect	link	setpgid	timer_getoverrun
creat	linkat	setsid	timer_gettime
dup	listen	setsockopt	timer_settime
dup2	lseek	setuid	times
execl	lstat	shutdown	umask
execle	mkdir	sigaction	uname
execv	mkdirat	sigaddset	unlink
execve	mkfifo	sigdelset	unlinkat
faccessat	mkfifoat	sigemptyset	utime
fchmod	mknod	sigfillset	utimensat
fchmodat	mknodat	sigismember	utimes
fchown	open	signal	wait
fchownat	openat	sigpause	waitpid
fcntl	pause	sigpending	write
fdatasync	pipe	sigprocmask	

图 8-33 异步信号安全的函数(来源：man 7 signal。数据来自 Linux Foundation)

信号处理程序中产生输出唯一安全的方法是使用 `write` 函数(见 10.1 节)。特别地,调用 `printf` 或 `sprintf` 是不安全的。为了绕开这个不幸的限制,我们开发一些安全的函数,称为 SIO(安全的 I/O)包,可以用来在信号处理程序中打印简单的消息。

```
#include "csapp.h"
```

```
ssize_t sio_putl(long v);
ssize_t sio_puts(char s[]);
```

返回: 如果成功则为传送的字节数, 如果出错, 则为 -1。

```
void sio_error(char s[]);
```

返回: 空。

`sio_putl` 和 `sio_puts` 函数分别向标准输出传送一个 `long` 类型数和一个字符串。`sio_error` 函数打印一条错误消息并终止。

图 8-34 给出的是 SIO 包的实现, 它使用了 `csapp.c` 中两个私有的可重入函数。第 3 行的 `sio_strlen` 函数返回字符串 `s` 的长度。第 10 行的 `sio_ltoa` 函数基于来自 [61] 的 `ltoa` 函数, 把 `v` 转换成它的基 `b` 字符串表示, 保存在 `s` 中。第 17 行的 `_exit` 函数是 `exit` 的一个异步信号安全的变种。

code/src/csapp.c

```
1  ssize_t sio_puts(char s[]) /* Put string */
2  {
3      return write(STDOUT_FILENO, s, sio_strlen(s));
4  }
5
6  ssize_t sio_putl(long v) /* Put long */
7  {
8      char s[128];
9
10     sio_ltoa(v, s, 10); /* Based on K&R ltoa() */
11     return sio_puts(s);
12 }
13
14 void sio_error(char s[]) /* Put error message and exit */
15 {
16     sio_puts(s);
17     _exit(1);
18 }
```

code/src/csapp.c

图 8-34 信号处理程序的 SIO(安全 I/O)包

图 8-35 给出了图 8-30 中 SIGINT 处理程序的一个安全的版本。

code/ecf/sigintsafe.c

```
1  #include "csapp.h"
2
3  void sigint_handler(int sig) /* Safe SIGINT handler */
4  {
5      Sio_puts("Caught SIGINT!\n"); /* Safe output */
6      _exit(0);                      /* Safe exit */
7  }
```

code/ecf/sigintsafe.c

图 8-35 图 8-30 的 SIGINT 处理程序的一个安全版本

- G2. 保存和恢复 `errno`。许多 Linux 异步信号安全的函数都会在出错返回时设置 `errno`。在处理程序中调用这样的函数可能会干扰主程序中其他依赖于 `errno` 的部分。解决方法是在进入处理程序时把 `errno` 保存在一个局部变量中，在处理程序返回前恢复它。注意，只有在处理程序要返回时才有此必要。如果处理程序调用 `_exit` 终止该进程，那么就不需要这样做了。
- G3. 阻塞所有的信号，保护对共享全局数据结构的访问。如果处理程序和主程序或其他处理程序共享一个全局数据结构，那么在访问(读或者写)该数据结构时，你的处理程序和主程序应该暂时阻塞所有的信号。这条规则的原因是从主程序访问一个数据结构 d 通常需要一系列的指令，如果指令序列被访问 d 的处理程序中断，那么处理程序可能会发现 d 的状态不一致，得到不可预知的结果。在访问 d 时暂时阻塞信号保证了处理程序不会中断该指令序列。
- G4. 用 `volatile` 声明全局变量。考虑一个处理程序和一个 `main` 函数，它们共享一个全局变量 g 。处理程序更新 g ，`main` 周期性地读 g 。对于一个优化编译器而言，`main` 中 g 的值看上去从来没有变化过，因此使用缓存在寄存器中 g 的副本来满足对 g 的每次引用是很安全的。如果这样，`main` 函数可能永远都无法看到处理程序更新过的值。

可以用 `volatile` 类型限定符来定义一个变量，告诉编译器不要缓存这个变量。例如：

```
volatile int g;
```

`volatile` 限定符强迫编译器每次在代码中引用 g 时，都要从内存中读取 g 的值。一般来说，和其他所有共享数据结构一样，应该暂时阻塞信号，保护每次对全局变量的访问。

- G5. 用 `sig_atomic_t` 声明标志。在常见的处理程序设计中，处理程序会写全局标志来记录收到了信号。主程序周期性地读这个标志，响应信号，再清除该标志。对于通过这种方式来共享的标志，C 提供一种整型数据类型 `sig_atomic_t`，对它的读和写保证会是原子的(不可中断的)，因为可以用一条指令来实现它们：

```
volatile sig_atomic_t flag;
```

因为它们是不可中断的，所以可以安全地读和写 `sig_atomic_t` 变量，而不需要暂时阻塞信号。注意，这里对原子性的保证只适用于单个的读和写，不适用于像 `flag++` 或 `flag=flag+10` 这样的更新，它们可能需要多条指令。

要记住我们这里讲述的规则是保守的，也就是说它们不总是严格必需的。例如，如果你知道处理程序绝对不会修改 `errno`，那么就不需要保存和恢复 `errno`。或者如果你可以证明 `printf` 的实例都不会被处理程序中断，那么在处理程序中调用 `printf` 就是安全的。对共享全局数据结构的访问也是同样。不过，一般来说这种断言很难证明。所以我们建议你采用保守的方法，遵循这些规则，使得处理程序尽可能简单，调用安全函数，保存和恢复 `errno`，保护对共享数据结构的访问，并使用 `volatile` 和 `sig_atomic_t`。

2. 正确的信号处理

信号的一个与直觉不符的方面是未处理的信号是不排队的。因为 `pending` 位向量中每种类型的信号只对应有一位，所以每种类型最多只能有一个未处理的信号。因此，如果两个类型 k 的信号发送给一个目的进程，而因为目的进程当前正在执行信号 k 的处理程序，所以信号 k 被阻塞了，那么第二个信号就简单地被丢弃了；它不会排队。关键思想是如果存在一个未处理的信号就表明至少有一个信号到达了。

要了解这样会如何影响正确性，来看一个简单的应用，它本质上类似于像 shell 和 Web 服务器这样的真实程序。基本的结构是父进程创建一些子进程，这些子进程各自独立运行一段时间，然后终止。父进程必须回收子进程以避免在系统中留下僵死进程。但是我们还希望父进程能够在子进程运行时自由地去做其他的工作。所以，我们决定用 SIGCHLD 处理程序来回收子进程，而不是显式地等待子进程终止。（回想一下，只要有一个子进程终止或者停止，内核就会发送一个 SIGCHLD 信号给父进程。）

图 8-36 展示了我们的初次尝试。父进程设置了一个 SIGCHLD 处理程序，然后创建

```

code/ecf/signall.c
1  /* WARNING: This code is buggy! */
2
3  void handler1(int sig)
4  {
5      int olderrno = errno;
6
7      if ((waitpid(-1, NULL, 0)) < 0)
8          sio_error("waitpid error");
9      Sio_puts("Handler reaped child\n");
10     Sleep(1);
11     errno = olderrno;
12 }
13
14 int main()
15 {
16     int i, n;
17     char buf[MAXBUF];
18
19     if (signal(SIGCHLD, handler1) == SIG_ERR)
20         unix_error("signal error");
21
22     /* Parent creates children */
23     for (i = 0; i < 3; i++) {
24         if (Fork() == 0) {
25             printf("Hello from child %d\n", (int)getpid());
26             exit(0);
27         }
28     }
29
30     /* Parent waits for terminal input and then processes it */
31     if ((n = read(STDIN_FILENO, buf, sizeof(buf))) < 0)
32         unix_error("read");
33
34     printf("Parent processing input\n");
35     while (1)
36         ;
37
38     exit(0);
39 }
code/ecf/signall.c

```

图 8-36 signall: 这个程序是有缺陷的，因为它假设信号是排队

了 3 个子进程。同时，父进程等待来自终端的一个输入行，随后处理它。这个处理被模型化为一个无限循环。当每个子进程终止时，内核通过发送一个 SIGCHLD 信号通知父进程。父进程捕获这个 SIGCHLD 信号，回收一个子进程，做一些其他的清理工作（模型化为 sleep 语句），然后返回。

图 8-36 中的 signal1 程序看起来相当简单。然而，当在 Linux 系统上运行它时，我们得到如下输出：

```
linux> ./signal1
Hello from child 14073
Hello from child 14074
Hello from child 14075
Handler reaped child
Handler reaped child
CR
Parent processing input
```

从输出中我们注意到，尽管发送了 3 个 SIGCHLD 信号给父进程，但是其中只有两个信号被接收了，因此父进程只是回收了两个子进程。如果挂起父进程，我们看到，实际上子进程 14075 没有被回收，它成了一个僵死进程（在 ps 命令的输出中由字符串 “defunct” 表明）：

```
Ctrl+Z
Suspended
linux> ps t
  PID TTY          STAT       TIME COMMAND
  :
  :
14072 pts/3        T           0:02 ./signal1
14075 pts/3        Z           0:00 [signal1] <defunct>
14076 pts/3        R+          0:00 ps t
```

哪里出错了呢？问题就在于我们的代码没有解决信号不会排队等待这样的情况。所发生的情况是：父进程接收并捕获了第一个信号。当处理程序还在处理第一个信号时，第二个信号就传送并添加到了待处理信号集合里。然而，因为 SIGCHLD 信号被 SIGCHLD 处理程序阻塞了，所以第二个信号就不会被接收。此后不久，就在处理程序还在处理第一个信号时，第三个信号到达了。因为已经有了一个待处理的 SIGCHLD，第三个 SIGCHLD 信号会被丢弃。一段时间之后，处理程序返回，内核注意到有一个待处理的 SIGCHLD 信号，就迫使父进程接收这个信号。父进程捕获这个信号，并第二次执行处理程序。在处理程序完成对第二个信号的处理之后，已经没有待处理的 SIGCHLD 信号了，而且也绝不会再有，因为第三个 SIGCHLD 的所有信息都已经丢失了。由此得到的重要教训是，不可以用信号来对其他进程中发生的事件计数。

为了修正这个问题，我们必须回想一下，存在一个待处理的信号只是暗示自进程最后一次收到一个信号以来，至少已经有一个这种类型的信号被发送了。所以我们必须修改 SIGCHLD 的处理程序，使得每次 SIGCHLD 处理程序被调用时，回收尽可能多的僵死子进程。图 8-37 展示了修改后的 SIGCHLD 处理程序。

当我们在 Linux 系统上运行 signal2 时，它现在可以正确地回收所有的僵死子进程了：

```
linux> ./signal2
Hello from child 15237
```

```

Hello from child 15238
Hello from child 15239
Handler reaped child
Handler reaped child
Handler reaped child
CR
Parent processing input

```

code/ecf/signal2.c

```

1 void handler2(int sig)
2 {
3     int olderrno = errno;
4
5     while (waitpid(-1, NULL, 0) > 0) {
6         Sio_puts("Handler reaped child\n");
7     }
8     if (errno != ECHILD)
9         Sio_error("waitpid error");
10    Sleep(1);
11    errno = olderrno;
12 }

```

code/ecf/signal2.c

图 8-37 signal2: 图 8-36 的一个改进版本, 它能够正确解决信号不会排队等待的情况

练习题 8.8 下面这个程序的输出是什么?

code/ecf/signalprob0.c

```

1 volatile long counter = 2;
2
3 void handler1(int sig)
4 {
5     sigset_t mask, prev_mask;
6
7     Sigfillset(&mask);
8     Sigprocmask(SIG_BLOCK, &mask, &prev_mask); /* Block sigs */
9     Sio_putl(--counter);
10    Sigprocmask(SIG_SETMASK, &prev_mask, NULL); /* Restore sigs */
11
12    _exit(0);
13 }
14
15 int main()
16 {
17     pid_t pid;
18     sigset_t mask, prev_mask;
19
20     printf("%ld", counter);
21     fflush(stdout);
22
23     signal(SIGUSR1, handler1);
24     if ((pid = Fork()) == 0) {

```



```

25         while(1) {};
26     }
27     Kill(pid, SIGUSR1);
28     Waitpid(-1, NULL, 0);
29
30     Sigfillset(&mask);
31     Sigprocmask(SIG_BLOCK, &mask, &prev_mask); /* Block sigs */
32     printf("%ld", ++counter);
33     Sigprocmask(SIG_SETMASK, &prev_mask, NULL); /* Restore sigs */
34
35     exit(0);
36 }

```

— code/ecf/signalprob0.c

3. 可移植的信号处理

Unix 信号处理的另一个缺陷在于不同的系统有不同的信号处理语义。例如：

- signal 函数的语义各有不同。有些老的 Unix 系统在信号 k 被处理程序捕获之后就将对信号 k 的反应恢复到默认值。在这些系统上，每次运行之后，处理程序必须调用 signal 函数，显式地重新设置它自己。
- 系统调用可以被中断。像 read、write 和 accept 这样的系统调用潜在地会阻塞进程一段较长的时间，称为慢速系统调用。在某些较早版本的 Unix 系统中，当处理程序捕获到一个信号时，被中断的慢速系统调用在信号处理程序返回时不再继续，而是立即返回给用户一个错误条件，并将 errno 设置为 EINTR。在这些系统上，程序员必须包括手动重启被中断的系统调用的代码。

要解决这些问题，Posix 标准定义了 sigaction 函数，它允许用户在设置信号处理时，明确指定他们想要的信号处理语义。

```

#include <signal.h>

int sigaction(int signum, struct sigaction *act,
              struct sigaction *oldact);

```

返回：若成功则为 0，若出错则为 -1。

sigaction 函数运用并不广泛，因为它要求用户设置一个复杂结构的条目。一个更简洁的方式，最初是由 W. Richard Stevens 提出的[110]，就是定义一个包装函数，称为 Signal，它调用 sigaction。图 8-38 给出了 Signal 的定义，它的调用方式与 signal 函数的调用方式一样。

Signal 包装函数设置了一个信号处理程序，其信号处理语义如下：

- 只有这个处理程序当前正在处理的那种类型的信号被阻塞。
- 和所有信号实现一样，信号不会排队等待。
- 只要可能，被中断的系统调用会自动重启。
- 一旦设置了信号处理程序，它就会一直保持，直到 Signal 带着 handler 参数为 SIG_IGN 或者 SIG_DFL 被调用。

我们在所有的代码中实现 Signal 包装函数。

8.5.6 同步流以避免讨厌的并发错误

如何编写读写相同存储位置的并发程序的问题，困扰着数代计算机科学家。一般而

言，流可能交错的数量与指令的数量呈指数关系。这些交错中的一些会产生正确的结果，而有些则不会。基本的问题是以某种方式同步并发流，从而得到最大的可行的交错的集合，每个可行的交错都能得到正确的结果。

code/src/csapp.c

```

1 handler_t *Signal(int signum, handler_t *handler)
2 {
3     struct sigaction action, old_action;
4
5     action.sa_handler = handler;
6     sigemptyset(&action.sa_mask); /* Block sigs of type being handled */
7     action.sa_flags = SA_RESTART; /* Restart syscalls if possible */
8
9     if (sigaction(signum, &action, &old_action) < 0)
10         unix_error("Signal error");
11     return (old_action.sa_handler);
12 }

```

code/src/csapp.c

图 8-38 Signal: sigaction 的一个包装函数，它提供在 Posix 兼容系统上的可移植的信号处理

并发编程是一个很深且很重要的问题，我们将在第 12 章中更详细地讨论。不过，在本章中学习的有关异常控制流的知识，可以让你感觉一下与并发相关的有趣的智力挑战。例如，考虑图 8-39 中的程序，它总结了一个典型的 Unix shell 的结构。父进程在一个全局作业列表中记录着它的当前子进程，每个作业一个条目。addjob 和 deletejob 函数分别向这个作业列表添加和从中删除作业。

当父进程创建一个新的子进程后，它就把这个子进程添加到作业列表中。当父进程在 SIGCHLD 处理程序中回收一个终止的（僵死）子进程时，它就从作业列表中删除这个子进程。

乍一看，这段代码是对的。不幸的是，可能发生下面这样的事件序列：

- 1) 父进程执行 fork 函数，内核调度新创建的子进程运行，而不是父进程。
- 2) 在父进程能够再次运行之前，子进程就终止，并且变成一个僵死进程，使得内核传递一个 SIGCHLD 信号给父进程。
- 3) 后来，当父进程再次变成可运行但又在它执行之前，内核注意到有未处理的 SIGCHLD 信号，并通过在父进程中运行处理程序接收这个信号。
- 4) 信号处理程序回收终止的子进程，并调用 deletejob，这个函数什么也不做，因为父进程还没有把该子进程添加到列表中。
- 5) 在处理程序运行完毕后，内核运行父进程，父进程从 fork 返回，通过调用 addjob 错误地把（不存在的）子进程添加到作业列表中。

因此，对于父进程的 main 程序和信号处理流的某些交错，可能会在 addjob 之前调用 deletejob。这导致作业列表中出现一个不正确的条目，对应于一个不再存在而且永远也不会被删除的作业。另一方面，也有一些交错，事件按照正确的顺序发生。例如，如果在 fork 调用返回时，内核刚好调度父进程而不是子进程运行，那么父进程就会正确地把子进程添加到作业列表中，然后子进程终止，信号处理函数把该作业从列表中删除。

这是一个称为竞争(race)的经典同步错误的示例。在这个情况中，main 函数中调用 addjob 和处理程序中调用 deletejob 之间存在竞争。如果 addjob 赢得进展，那么结果

就是正确的。如果它没有，那么结果就是错误的。这样的错误非常难以调试，因为几乎不可能测试所有的交错。你可能运行这段代码十亿次，也没有一次错误，但是下一次测试却导致引发竞争的交错。

```

1  /* WARNING: This code is buggy! */
2  void handler(int sig)
3  {
4      int olderrno = errno;
5      sigset_t mask_all, prev_all;
6      pid_t pid;
7
8      Sigfillset(&mask_all);
9      while ((pid = waitpid(-1, NULL, 0)) > 0) { /* Reap a zombie child */
10         Sigprocmask(SIG_BLOCK, &mask_all, &prev_all);
11         deletejob(pid); /* Delete the child from the job list */
12         Sigprocmask(SIG_SETMASK, &prev_all, NULL);
13     }
14     if (errno != ECHILD)
15         Sio_error("waitpid error");
16     errno = olderrno;
17 }
18
19 int main(int argc, char **argv)
20 {
21     int pid;
22     sigset_t mask_all, prev_all;
23
24     Sigfillset(&mask_all);
25     Signal(SIGCHLD, handler);
26     initjobs(); /* Initialize the job list */
27
28     while (1) {
29         if ((pid = Fork()) == 0) { /* Child process */
30             Execve("/bin/date", argv, NULL);
31         }
32         Sigprocmask(SIG_BLOCK, &mask_all, &prev_all); /* Parent process */
33         addjob(pid); /* Add the child to the job list */
34         Sigprocmask(SIG_SETMASK, &prev_all, NULL);
35     }
36     exit(0);
37 }

```

code/ecf/procmask1.c

图 8-39 一个具有细微同步错误的 shell 程序。如果子进程在父进程能够开始运行前就结束了，那么 `addjob` 和 `deletejob` 会以错误的方式被调用

图 8-40 展示了消除图 8-39 中竞争的一种方法。通过在调用 `fork` 之前，阻塞 `SIGCHLD` 信号，然后在调用 `addjob` 之后取消阻塞这些信号，我们保证了在子进程被添加到作业列表之后回收该子进程。注意，子进程继承了它们父进程的被阻塞集合，所以我们必须在调用 `execve` 之前，小心地解除子进程中阻塞的 `SIGCHLD` 信号。

code/ecf/procmask2.c

```

1 void handler(int sig)
2 {
3     int olderrno = errno;
4     sigset_t mask_all, prev_all;
5     pid_t pid;
6
7     Sigfillset(&mask_all);
8     while ((pid = waitpid(-1, NULL, 0)) > 0) { /* Reap a zombie child */
9         Sigprocmask(SIG_BLOCK, &mask_all, &prev_all);
10        deletejob(pid); /* Delete the child from the job list */
11        Sigprocmask(SIG_SETMASK, &prev_all, NULL);
12    }
13    if (errno != ECHILD)
14        Sio_error("waitpid error");
15    errno = olderrno;
16 }
17
18 int main(int argc, char **argv)
19 {
20     int pid;
21     sigset_t mask_all, mask_one, prev_one;
22
23     Sigfillset(&mask_all);
24     Sigemptyset(&mask_one);
25     Sigaddset(&mask_one, SIGCHLD);
26     Signal(SIGCHLD, handler);
27     initjobs(); /* Initialize the job list */
28
29     while (1) {
30         Sigprocmask(SIG_BLOCK, &mask_one, &prev_one); /* Block SIGCHLD */
31         if ((pid = Fork()) == 0) { /* Child process */
32             Sigprocmask(SIG_SETMASK, &prev_one, NULL); /* Unblock SIGCHLD */
33             Execve("/bin/date", argv, NULL);
34         }
35         Sigprocmask(SIG_BLOCK, &mask_all, NULL); /* Parent process */
36         addjob(pid); /* Add the child to the job list */
37         Sigprocmask(SIG_SETMASK, &prev_one, NULL); /* Unblock SIGCHLD */
38     }
39     exit(0);
40 }

```

code/ecf/procmask2.c

图 8-40 用 sigprocmask 来同步进程。在这个例子中，父进程保证在相应的 deletejob 之前执行 addjob

8.5.7 显式地等待信号

有时候主程序需要显式地等待某个信号处理程序运行。例如，当 Linux shell 创建一个前台作业时，在接收下一条用户命令之前，它必须等待作业终止，被 SIGCHLD 处理程序回收。

图 8-41 给出了一个基本的思路。父进程设置 SIGINT 和 SIGCHLD 的处理程序，然后

进入一个无限循环。它阻塞 SIGCHLD 信号，避免 8.5.6 节中讨论过的父进程和子进程之间的竞争。创建了子进程之后，把 pid 重置为 0，取消阻塞 SIGCHLD，然后以循环的方式等待 pid 变为非零。子进程终止后，处理程序回收它，把它非零的 PID 赋值给全局 pid 变量。这会终止循环，父进程继续其他的工作，然后开始下一次迭代。

code/ecf/waitforsignal.c

```
1  #include "csapp.h"
2
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)
6  {
7      int olderrno = errno;
8      pid = waitpid(-1, NULL, 0);
9      errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
15
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         Sigprocmask(SIG_BLOCK, &mask, &prev); /* Block SIGCHLD */
27         if (Fork() == 0) /* Child */
28             exit(0);
29
30         /* Parent */
31         pid = 0;
32         Sigprocmask(SIG_SETMASK, &prev, NULL); /* Unblock SIGCHLD */
33
34         /* Wait for SIGCHLD to be received (wasteful) */
35         while (!pid)
36             ;
37
38         /* Do some work after receiving SIGCHLD */
39         printf(".");
40     }
41     exit(0);
42 }
```

code/ecf/waitforsignal.c

图 8-11 用循环来等待信号。这段代码正确，但循环是一种浪费

当这段代码正确执行的时候，循环在浪费处理器资源。我们可能会想要修补这个问题，在循环体内插入 `pause`：

```
while (!pid) /* Race! */
    pause();
```

注意，我们仍然需要一个循环，因为收到一个或多个 `SIGINT` 信号，`pause` 会被中断。不过，这段代码有很严重的竞争条件：如果在 `while` 测试后和 `pause` 之前收到 `SIGCHLD` 信号，`pause` 会永远睡眠。

另一个选择是用 `sleep` 替换 `pause`：

```
while (!pid) /* Too slow! */
    sleep(1);
```

当这段代码正确执行时，它太慢了。如果在 `while` 之后 `pause` 之前收到信号，程序必须等相当长的一段时间才会再次检查循环的终止条件。使用像 `nanosleep` 这样更高精度的休眠函数也是不可接受的，因为没有很好的方法来确定休眠的间隔。间隔太小，循环会太浪费。间隔太大，程序又会太慢。

合适的解决方法是使用 `sigsuspend`。

```
#include <signal.h>

int sigsuspend(const sigset_t *mask);
```

返回：-1。

`sigsuspend` 函数暂时用 `mask` 替换当前的阻塞集合，然后挂起该进程，直到收到一个信号，其行为要么是运行一个处理程序，要么是终止该进程。如果它的行为是终止，那么该进程不从 `sigsuspend` 返回就直接终止。如果它的行为是运行一个处理程序，那么 `sigsuspend` 从处理程序返回，恢复调用 `sigsuspend` 时原有的阻塞集合。

`sigsuspend` 函数等价于下述代码的原子（不可中断的）版本：

```
1    sigprocmask(SIG_SETMASK, &mask, &prev);
2    pause();
3    sigprocmask(SIG_SETMASK, &prev, NULL);
```

原子属性保证对 `sigprocmask`（第 1 行）和 `pause`（第 2 行）的调用总是一起发生的，不会被中断。这样就消除了潜在的竞争，即在调用 `sigprocmask` 之后但在调用 `pause` 之前收到了一个信号。

图 8-42 展示了如何使用 `sigsuspend` 来替代图 8-41 中的循环。在每次调用 `sigsuspend` 之前，都要阻塞 `SIGCHLD`。`sigsuspend` 会暂时取消阻塞 `SIGCHLD`，然后休眠，直到父进程捕获信号。在返回之前，它会恢复原始的阻塞集合，又再次阻塞 `SIGCHLD`。如果父进程捕获一个 `SIGINT` 信号，那么循环测试成功，下一次迭代又再次调用 `sigsuspend`。如果父进程捕获一个 `SIGCHLD`，那么循环测试失败，会退出循环。此时，`SIGCHLD` 是被阻塞的，所以我们可以可选地取消阻塞 `SIGCHLD`。在真实的有后台作业需要回收的 shell 中这样做可能会有用处。

`sigsuspend` 版本比起原来的循环版本不那么浪费，避免了引入 `pause` 带来的竞争，又比 `sleep` 更有效率。

code/ecf/sigsuspend.c

```
1  #include "csapp.h"
2
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)
6  {
7      int olderrno = errno;
8      pid = Waitpid(-1, NULL, 0);
9      errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
15
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         Sigprocmask(SIG_BLOCK, &mask, &prev); /* Block SIGCHLD */
27         if (Fork() == 0) /* Child */
28             exit(0);
29
30         /* Wait for SIGCHLD to be received */
31         pid = 0;
32         while (!pid)
33             sigsuspend(&prev);
34
35         /* Optionally unblock SIGCHLD */
36         Sigprocmask(SIG_SETMASK, &prev, NULL);
37
38         /* Do some work after receiving SIGCHLD */
39         printf(".");
40     }
41     exit(0);
42 }
```

code/ecf/sigsuspend.c

图 8-42 用 sigsuspend 来等待信号

8.6 非本地跳转

C 语言提供了一种用户级异常控制流形式，称为非本地跳转(nonlocal jump)，它将控制直接从一个函数转移到另一个当前正在执行的函数，而不需要经过正常的调用-返回序